COMPUTER SCIENCE / 9618

Pseudocode Workbook

From confident foundations to advanced algorithm practice

Independent practice for Cambridge International Computer Science 9618

12 core tasks	4 advanced tasks
Worked solutions and test cases	Suggested marks and teacher notes
Printable writing space	Editable source included

Name: _____

Class: _____ Date: _____

November 2026 • Original learning resource

An independent learning resource. Questions, explanations and mark allocations were written for this workbook. This publication is not endorsed by Cambridge International and is not a reproduction of the Scribd workbook.

LEARN / PRACTISE / REVIEW

How to use this workbook

Read the reference pages, trace the worked example, then attempt each task before consulting its solution. Write on the lines provided or use an additional sheet for longer algorithms.

Section	Pages	Purpose
Notation and control flow	3–4	Read and write consistent pseudocode
Worked trace and data structures	5–6	See how state changes during execution
Core practice: tasks 1–12	7–12	Write, test and improve your algorithms
Core worked solutions	13–24	Compare logic and identify missed cases
Advanced practice: tasks 13–16	25–28	Recursion, stack, queue and text files
Advanced worked solutions	29–32	Review state, boundaries and termination
Teacher guidance and references	33–34	Assess, reteach and extend

A useful practice cycle

1. State the inputs, outputs and assumptions.
2. Choose the data and control structures.
3. Write an algorithm with clear indentation.
4. Trace a normal case and a boundary case.
5. Compare with the suggested solution and explain any differences.

Levels and scope

Tasks 1–12 build from introductory programming towards AS algorithm work. Tasks 13–16 are advanced practice, including A2-style data-structure operations. The workbook is a focused practice resource rather than a complete course or a past-paper collection.

Core tasks: 84 suggested marks. Advanced tasks: 34 suggested marks. Total: 118. These allocations are for classroom feedback; they are not official Cambridge mark schemes.

Conventions

Inputs have the stated types. Validate only when a question asks for it. Supplied arrays and global state already exist; declarations marked as supplied are explanatory comments. Array bounds are inclusive. Each test row represents a separate run unless stated otherwise.

LEARN / PRACTISE / REVIEW

Notation at a glance

Use descriptive identifiers, capitalised keywords and indentation that shows which statements belong together. The following are syntax reminders; the examples throughout this workbook are original.

Idea	Example / meaning
Declaration	DECLARE Balance : REAL
Constant	CONSTANT Capacity = 12
Assignment	Balance ← Balance + 2.50
Input / output	INPUT Balance; OUTPUT Balance (write on separate lines)
Integer / real	INTEGER stores whole numbers; REAL can hold fractions
Text / truth	STRING for text; CHAR for a character; BOOLEAN for TRUE or FALSE
Comparison	=, <>, <, <=, >, >= produce a Boolean result
Logic	AND, OR, NOT combine or invert Boolean values
Division	17 / 5 gives 3.4; 17 DIV 5 gives 3; 17 MOD 5 gives 2
Joining strings	"Unit" & "-" & "4" produces "Unit-4"
Comment	// Explain an assumption or purpose

Assignment and comparison have different jobs

```
DECLARE Points : INTEGER
Points ← 7
IF Points = 7 THEN
   OUTPUT "Target reached"
ENDIF
```

The arrow changes Points. The equality test asks a question about its current value. A variable must have a value before an expression uses it.

String helpers used here

LENGTH(S) counts characters. MID(S, Start, Count) extracts Count characters starting at the 1-based position Start. Each string task states the helpers it allows.

LEARN / PRACTISE / REVIEW

Choose the control structure

Choose a structure based on what controls the next step: a condition, a fixed count, or the arrival of a stopping value.

Structure	Choose it when...	Boundary to check
IF / ELSE	One of two actions is needed	Equality at the threshold
FOR / NEXT	The iteration range is known	First and last iteration
WHILE	The body may run zero times	Condition false initially
REPEAT / UNTIL	The body must run at least once	Condition true after first run

Count a known range

```
DECLARE Step : INTEGER
FOR Step ← 2 TO 8 STEP 2
   OUTPUT Step
NEXT Step
```

This outputs 2, 4, 6, 8. NEXT Step closes the FOR loop. STEP is optional when the increment is 1.

Repeat until an input is valid

```
DECLARE Size : INTEGER
REPEAT
   INPUT Size
UNTIL (Size >= 1) AND (Size <= 6)
OUTPUT Size
```

The condition means stop when Size is within the inclusive range. Inputs 0, 7, 6 require three attempts and produce 6.

Review before you move on

What happens if the first input is valid? Why would a WHILE loop need an initial value before checking it? What statement ensures progress towards termination?

LEARN / PRACTISE / REVIEW

Worked example: filter and total

Read INTEGER values until 0. Add only positive values and count how many were added. Output Total and Count. Negative values are ignored.

```
DECLARE Value, Total, Count : INTEGER
Total ← 0
Count ← 0
INPUT Value
WHILE Value <> 0
   IF Value > 0 THEN
      Total ← Total + Value
      Count ← Count + 1
   ENDIF
   INPUT Value
ENDWHILE
OUTPUT Total, Count
```

Trace for inputs 6, -2, 9, 0

Input	Action	Total after action	Count after action
—	Initialise	0	0
6	Include	6	1
-2	Ignore	6	1
9	Include	15	2
0	Stop before body	15	2

Why this works

After each processed value, Total is the sum of the positive values seen so far and Count is their number. Initialising both to zero makes that statement true before any values are processed. The sentinel is tested before the body, so it contributes nothing.

Check another route

For inputs -3, -1, 0, both outputs are 0. If the first input is 0, the loop runs zero times and both outputs are also 0. The final INPUT inside the body is essential: it moves the algorithm towards the sentinel.

LEARN / PRACTISE / REVIEW

Arrays, modules and file state

An algorithm becomes easier to reason about when its data has clear bounds and each module has one defined responsibility.

Declare the shape, then respect the bounds

```
DECLARE Levels : ARRAY[1:12] OF INTEGER
DECLARE Grid : ARRAY[1:3,1:4] OF INTEGER
DECLARE Index : INTEGER
FOR Index ← 1 TO 12
   Levels[Index] ← 0
NEXT Index
```

An array's capacity and its used length may differ. If N entries are used in Levels, process 1 to N rather than all 12. A zero used length means there is no valid used element to inspect.

Return a value or perform an action

```
FUNCTION AddFee(Price : REAL) RETURNS REAL
   RETURN Price + 1.25
ENDFUNCTION

PROCEDURE ShowReceipt(Amount : REAL)
   OUTPUT "Amount: ", Amount
ENDPROCEDURE

CALL ShowReceipt(AddFee(8.00))
```

A function call belongs inside an expression; a procedure is invoked using CALL. A BYREF parameter allows the called module to change the caller's variable. Otherwise parameters here are passed by value.

File lifecycle

Open a file in its required mode, read or write, then close it. For sequential reading, test EOF before READFILE. READFILE gives a STRING for one line. WRITE mode replaces an existing file; APPEND adds to its end.

State safety

Check empty and full conditions before using a stack or queue slot. Test a string's length before extracting a fixed prefix. Place potentially unsafe accesses inside a guarded branch rather than assuming AND will skip an operand.

PRACTISE / WRITE BEFORE READING THE SOLUTION

Core practice / 01–02

01 School trip budget [5 marks]

Sequence • REAL arithmetic

Read a positive integer Students and a non-negative REAL TicketPrice. A coach costs 180.00 in total. Output the total trip cost and the cost per student, sharing the coach cost equally. You may assume the inputs have the stated types and ranges. Do not round the results.

Check: 30 students; ticket 12.50 → Total 555.00; per student 18.50

Plan and pseudocode (continue on another sheet if needed):

02 Parcel charge bands [6 marks]

Selection • boundary values

Read a REAL Mass. If Mass is zero or negative, output Invalid mass. Otherwise charge 3.00 for a mass up to and including 1 kg, 5.50 for a mass above 1 kg and up to and including 5 kg, and 9.00 above 5 kg. Output only the applicable charge or the invalid message.

Check: Mass: 0, 1, 1.01 → Invalid mass; 3.00; 5.50

Plan and pseudocode (continue on another sheet if needed):

PRACTISE / WRITE BEFORE READING THE SOLUTION

Core practice / 03–04

03 Three chances to unlock [7 marks]

WHILE • counters • Boolean flags

A practice lock accepts the STRING code 2468. Read at most three code attempts. Stop asking as soon as an attempt is correct. Output Access granted after a correct attempt, or Locked after three incorrect attempts. Treat the code as text; each INPUT supplies one attempt.

Check: 2468 → One input; Access granted

Plan and pseudocode (continue on another sheet if needed):

04 Temperature log [8 marks]

Sentinel input • totals • empty data

Read INTEGER temperatures until 999 is entered. The value 999 is a sentinel and must not be processed. Output the average of the readings, or No readings if the sentinel is the first input. Valid readings may be positive, zero, or negative. Assume every non-sentinel input is a valid temperature.

Check: -4, 0, 10, 999 → Average 2.0

Plan and pseudocode (continue on another sheet if needed):

PRACTISE / WRITE BEFORE READING THE SOLUTION

Core practice / 05–06

05 Seats still available [6 marks]

Arrays • counting • inclusive ranges

Read eight INTEGER values into FreeSeats[1:8]. Each value is the number of free seats on one minibus and is guaranteed to be from 0 to 15 inclusive. Output how many minibuses have at least four free seats, and output the total number of free seats across all eight minibuses.

Check: 0, 4, 7, 3, 15, 4, 1, 2 → 4 qualifying buses; 36 free seats

Plan and pseudocode (continue on another sheet if needed):

06 Mirror a message [6 marks]

Strings • reverse traversal

Read a STRING Message, which may be empty. Build and output a second string containing the characters of Message in reverse order. Use LENGTH(S) for the number of characters and MID(S, Start, Count) for a substring; positions start at 1. Use & to concatenate strings. Do not call a built-in reverse function.

Check: Code! → !edoC

Plan and pseudocode (continue on another sheet if needed):

PRACTISE / WRITE BEFORE READING THE SOLUTION

Core practice / 07–08

07 Busiest activity row [8 marks]

2D arrays • nested loops • ties

Counts is an already populated INTEGER array with bounds [1:3,1:4]. Values are non-negative. Each row represents one activity and each column one session. Output each row total, then the row number with the largest total. If rows tie, choose the smallest row number. You do not need to read the array.

Check: Rows: [1,2,3,4]; [5,0,5,0]; [0,2,0,1] → Totals 10, 10, 3; best row 1

Plan and pseudocode (continue on another sheet if needed):

08 First matching badge [7 marks]

Linear search • duplicates • not found

Badges is an already populated STRING array with bounds [1:40]. N is an INTEGER from 0 to 40 giving the number of used entries. Read Target and output the index of its first occurrence within positions 1 to N, or 0 if it is absent. Repeated badge values are possible. Do not examine entries beyond N.

Check: Used badges A, C, A; Target A → 1

Plan and pseudocode (continue on another sheet if needed):

PRACTISE / WRITE BEFORE READING THE SOLUTION

Core practice / 09–10

09 Sort waiting times [8 marks]

Bubble sort • shrinking bounds

Times is an already populated INTEGER array with bounds [1:30]. N, from 1 to 30, gives its used length. Sort the used entries into ascending order using bubble sort. End early if a whole pass makes no swaps. Output each used value after sorting. Duplicate values must be retained.

Check: Used values 9, 3, 3, 1 → 1, 3, 3, 9

Plan and pseudocode (continue on another sheet if needed):

10 Find a sorted catalogue code [8 marks]

Binary search • integer midpoint

Codes is an INTEGER array with bounds [1:100]. Its N used entries are already sorted in ascending order and contain distinct values; N is from 0 to 100. Read Target and use binary search to output its index, or 0 if absent. Do not sort the array or use linear search.

Check: Codes 4, 9, 15, 22; Target 22 → 4

Plan and pseudocode (continue on another sheet if needed):

PRACTISE / WRITE BEFORE READING THE SOLUTION

Core practice / 11–12

11 A reusable common-factor function [7 marks]

Functions • MOD • Euclid's algorithm

Write a function CommonFactor(A : INTEGER, B : INTEGER) RETURNS INTEGER that returns the greatest common divisor of two positive integers. Use this method: repeatedly replace A by B and B by the remainder of the old A divided by the old B, stopping when B becomes zero. Also show a call for 84 and 30 that outputs the return value.

Check: 84, 30 → 6

Plan and pseudocode (continue on another sheet if needed):

12 Select the best project [8 marks]

Records • filtering • tie handling

Define a record type ProjectRecord with a STRING Title, an INTEGER Score, and a BOOLEAN Submitted. Projects is an already populated array of 20 such records; N, from 0 to 20, gives the used length. Output the title and score of the submitted project with the highest score, or No submissions. Scores are 0 to 100. On a tie, use the first project in array order.

Check: (Map,70,FALSE), (Game,60,TRUE), (Art,60,TRUE) → Game, 60

Plan and pseudocode (continue on another sheet if needed):

REVIEW / CORE WORKED SOLUTION

01 / School trip budget

Sequence • REAL arithmetic • 5 suggested marks

```
DECLARE Students : INTEGER
DECLARE TicketPrice, TotalCost, CostEach : REAL
INPUT Students
INPUT TicketPrice
TotalCost ← Students * TicketPrice + 180.00
CostEach ← TotalCost / Students
OUTPUT TotalCost, CostEach
```

The multiplication accounts for every ticket. The fixed coach charge is then added to the total. Division produces a REAL result; Students cannot be zero under the question's precondition.

Test input / state	Expected result
30 students; ticket 12.50	Total 555.00; per student 18.50
1 student; ticket 0.00	Total 180.00; per student 180.00

Suggested marking points	Marks
Suitable declarations and both inputs	1
Ticket cost multiplied by student count	1
Coach cost added once	1
Total divided by student count	1
Both results output	1

Common error: Dividing only the ticket price by the number of students gives the wrong share of the total.

REVIEW / CORE WORKED SOLUTION

02 / Parcel charge bands

Selection • boundary values • 6 suggested marks

```
DECLARE Mass : REAL
INPUT Mass
IF Mass <= 0 THEN
   OUTPUT "Invalid mass"
ELSE
   IF Mass <= 1 THEN
      OUTPUT 3.00
   ELSE
      IF Mass <= 5 THEN
         OUTPUT 5.50
      ELSE
         OUTPUT 9.00
      ENDIF
   ENDIF
ENDIF
```

Each ELSE removes a range already considered. By the time Mass <= 5 is tested, Mass is known to be greater than 1. The nested branches ensure exactly one output.

Test input / state	Expected result
Mass: 0, 1, 1.01	Invalid mass; 3.00; 5.50
Mass: 5, 5.01	5.50; 9.00

Suggested marking points	Marks
Read Mass with a suitable type	1
Reject zero and negative masses	1
Correct inclusive 1 kg boundary	1
Correct inclusive 5 kg boundary	1
Correct remaining charge	1
Exactly one appropriate output	1

Common error: Separate IF statements can print more than one price for the same mass.

REVIEW / CORE WORKED SOLUTION

03 / Three chances to unlock

WHILE • counters • Boolean flags • 7 suggested marks

```
DECLARE Attempt : STRING
DECLARE Attempts : INTEGER
DECLARE Unlocked : BOOLEAN
Attempts ← 0
Unlocked ← FALSE
WHILE (Attempts < 3) AND (Unlocked = FALSE)
   INPUT Attempt
   Attempts ← Attempts + 1
   IF Attempt = "2468" THEN
      Unlocked ← TRUE
   ENDIF
ENDWHILE
IF Unlocked THEN
   OUTPUT "Access granted"
ELSE
   OUTPUT "Locked"
ENDIF
```

The counter changes on every input, including a successful attempt. A successful third attempt is still accepted because the final message depends on Unlocked rather than the counter alone.

Test input / state	Expected result
2468	One input; Access granted
1111, 0000, 2468	Three inputs; Access granted
a, b, c	Three inputs; Locked

Suggested marking points	Marks
Counter and flag initialised	1
Loop enforces the three-attempt limit	1
Loop also stops after success	1
Input and counter update on each attempt	1
Compare with the correct STRING code	1
Update success flag	1
Correct final message	1

Common error: Using Attempts = 3 to decide failure rejects a correct third attempt.

REVIEW / CORE WORKED SOLUTION

04 / Temperature log

Sentinel input • totals • empty data • 8 suggested marks

```
DECLARE Reading, Total, Count : INTEGER
DECLARE Average : REAL
Total ← 0
Count ← 0
INPUT Reading
WHILE Reading <> 999
   Total ← Total + Reading
   Count ← Count + 1
   INPUT Reading
ENDWHILE
IF Count = 0 THEN
   OUTPUT "No readings"
ELSE
   Average ← Total / Count
   OUTPUT Average
ENDIF
```

Testing the sentinel before accumulation excludes it from both Total and Count. The empty-data branch avoids division by zero. Negative readings require no special case.

Test input / state	Expected result
-4, 0, 10, 999	Average 2.0
999	No readings
-8, 999	Average -8.0

Suggested marking points	Marks
Suitable declarations and zero initialisation	1
Initial input	1
Loop stops at 999	1
Accumulate valid values	1
Count valid values	1
Read the next value inside the loop	1
Handle zero readings	1
Calculate and output the average	1

Common error: Counting 999 as a reading changes both the numerator and the denominator.

REVIEW / CORE WORKED SOLUTION

05 / Seats still available

Arrays • counting • inclusive ranges • 6 suggested marks

```
DECLARE FreeSeats : ARRAY[1:8] OF INTEGER
DECLARE Index, Suitable, Total : INTEGER
Suitable ← 0
Total ← 0
FOR Index ← 1 TO 8
   INPUT FreeSeats[Index]
   Total ← Total + FreeSeats[Index]
   IF FreeSeats[Index] >= 4 THEN
      Suitable ← Suitable + 1
   ENDIF
NEXT Index
OUTPUT Suitable, Total
```

Every input contributes its seat value to Total. Only inputs meeting the inclusive threshold contribute one to Suitable. All eight array positions are visited exactly once.

Test input / state	Expected result
0, 4, 7, 3, 15, 4, 1, 2	4 qualifying buses; 36 free seats
0, 0, 0, 0, 0, 0, 0, 0	0 qualifying buses; 0 free seats

Suggested marking points	Marks
Correct array declaration	1
Both accumulators initialised	1
Loop reads all eight elements	1
Accumulate seat values	1
Count values >= 4	1
Output both results	1

Common error: Suitable must increase by one per qualifying bus, rather than by its seat count.

REVIEW / CORE WORKED SOLUTION

06 / Mirror a message

Strings • reverse traversal • 6 suggested marks

```
DECLARE Message, Mirrored : STRING
DECLARE Position : INTEGER
INPUT Message
Mirrored ← ""
FOR Position ← LENGTH(Message) TO 1 STEP -1
   Mirrored ← Mirrored & MID(Message, Position, 1)
NEXT Position
OUTPUT Mirrored
```

Each substring contains one character. For an empty input the starting position is zero, so the negative-step loop has no iterations and the result stays empty.

Test input / state	Expected result
Code!	!edoC
a	a
Empty string	Empty string

Suggested marking points	Marks
Suitable declarations and input	1
Empty result initialised	1
Start at the final character	1
Move backwards to position 1	1
Extract and concatenate one character	1
Output the reversed string	1

Common error: Using + instead of & treats string joining like arithmetic.

REVIEW / CORE WORKED SOLUTION

07 / Busiest activity row

2D arrays • nested loops • ties • 8 suggested marks

```
// Counts is supplied with bounds [1:3,1:4]
DECLARE Row, Column, RowTotal : INTEGER
DECLARE BestRow, BestTotal : INTEGER
BestRow ← 1
BestTotal ← -1
FOR Row ← 1 TO 3
   RowTotal ← 0
   FOR Column ← 1 TO 4
      RowTotal ← RowTotal + Counts[Row, Column]
   NEXT Column
   OUTPUT RowTotal
   IF RowTotal > BestTotal THEN
      BestTotal ← RowTotal
      BestRow ← Row
   ENDIF
NEXT Row
OUTPUT BestRow
```

Because all entries are non-negative, -1 is below every possible row total. Updating only for a strictly greater total keeps the earlier row in a tie.

Test input / state	Expected result
Rows: [1,2,3,4]; [5,0,5,0]; [0,2,0,1]	Totals 10, 10, 3; best row 1
All twelve values are 0	Totals 0, 0, 0; best row 1

Suggested marking points	Marks
Valid initial best row and total	1
Outer loop covers three rows	1
Reset RowTotal for each row	1
Inner loop covers four columns	1
Correct 2D indexing and accumulation	1
Output each row total	1
Update total and row while preserving ties	1
Output the winning row number	1

Common error: Resetting RowTotal before the outer loop produces running totals across rows.

REVIEW / CORE WORKED SOLUTION

08 / First matching badge

Linear search • duplicates • not found • 7 suggested marks

```
// Badges and N are supplied
DECLARE Target : STRING
DECLARE Index, Position : INTEGER
INPUT Target
Index ← 1
Position ← 0
WHILE (Index <= N) AND (Position = 0)
   IF Badges[Index] = Target THEN
      Position ← Index
   ENDIF
   Index ← Index + 1
ENDWHILE
OUTPUT Position
```

The loop condition contains no array access. Consequently no assumption about short-circuit Boolean evaluation is needed to keep the array access safe. For N = 0, the body is skipped.

Test input / state	Expected result
Used badges A, C, A; Target A	1
Used badges A, C, A; Target Z	0
N = 0; any Target	0

Suggested marking points	Marks
Read Target and initialise search state	1
Loop stays within used entries	1
Loop stops after the first match	1
Compare the current badge	1
Save its index	1
Advance the index	1
Correct output for found and absent targets	1

Common error: Continuing and overwriting Position on every match returns the last match.

REVIEW / CORE WORKED SOLUTION

09 / Sort waiting times

Bubble sort • shrinking bounds • 8 suggested marks

```
// Times and N are supplied
DECLARE Last, Index, Temp : INTEGER
DECLARE Swapped : BOOLEAN
Last ← N
Swapped ← TRUE
WHILE (Last > 1) AND Swapped
   Swapped ← FALSE
   FOR Index ← 1 TO Last - 1
      IF Times[Index] > Times[Index + 1] THEN
         Temp ← Times[Index]
         Times[Index] ← Times[Index + 1]
         Times[Index + 1] ← Temp
         Swapped ← TRUE
      ENDIF
   NEXT Index
   Last ← Last - 1
ENDWHILE
FOR Index ← 1 TO N
   OUTPUT Times[Index]
NEXT Index
```

Each pass moves the largest unsorted value to Last. The next pass excludes that position. A pass without swaps proves that the remaining prefix is in order.

Test input / state	Expected result
Used values 9, 3, 3, 1	1, 3, 3, 9
Used value 5	5; no comparison needed
Used values 1, 2, 3	1, 2, 3; first pass makes no swaps

Suggested marking points	Marks
Initialise pass state	1
Reset swap flag each pass	1
Correct adjacent comparison bounds	1
Compare in ascending-order direction	1
Correct three-assignment swap	1
Set flag after a swap	1
Shrink the bound and stop when sorted	1
Output all used entries	1

Common error: Comparing up to Last accesses Times[Last + 1], outside the active region.

REVIEW / CORE WORKED SOLUTION

10 / Find a sorted catalogue code

Binary search • integer midpoint • 8 suggested marks

```
// Codes and N are supplied
DECLARE Target, Low, High, Mid, Position : INTEGER
INPUT Target
Low ← 1
High ← N
Position ← 0
WHILE (Low <= High) AND (Position = 0)
   Mid ← (Low + High) DIV 2
   IF Codes[Mid] = Target THEN
      Position ← Mid
   ELSE
      IF Codes[Mid] < Target THEN
         Low ← Mid + 1
      ELSE
         High ← Mid - 1
      ENDIF
   ENDIF
ENDWHILE
OUTPUT Position
```

Low and High describe an inclusive search interval. Removing Mid after a failed comparison guarantees progress. An empty array starts with Low > High and needs no special array access.

Test input / state	Expected result
Codes 4, 9, 15, 22; Target 22	4
Codes 4, 9, 15, 22; Target 10	0
N = 0; any Target	0

Suggested marking points	Marks
Read target and initialise bounds	1
Initialise absent-result state	1
Correct loop termination	1
Integer midpoint	1
Record a successful match	1
Correct lower-bound update	1
Correct upper-bound update	1
Output found index or 0	1

Common error: Low ← Mid can repeat the same midpoint forever when the interval is small.

REVIEW / CORE WORKED SOLUTION

11 / A reusable common-factor function

Functions • MOD • Euclid's algorithm • 7 suggested marks

```
FUNCTION CommonFactor(A : INTEGER, B : INTEGER)
   RETURNS INTEGER
   DECLARE Remainder : INTEGER
   WHILE B <> 0
      Remainder ← A MOD B
      A ← B
      B ← Remainder
   ENDWHILE
   RETURN A
ENDFUNCTION

OUTPUT CommonFactor(84, 30)
```

The nonzero remainders decrease until B becomes zero. At that point A holds the greatest common divisor. The parameters are passed by value, so this function does not change the caller's variables.

Test input / state	Expected result
84, 30	6
7, 7	7
13, 5	1

Suggested marking points	Marks
Function header with typed parameters and return type	1
Loop stops when B is zero	1
Calculate A MOD B before overwriting	1
Replace A with old B	1
Replace B with the saved remainder	1
RETURN A	1
Correct output of a function call	1

Common error: OUTPUT inside the function is not a substitute for returning a value to its caller.

REVIEW / CORE WORKED SOLUTION

12 / Select the best project

Records • filtering • tie handling • 8 suggested marks

```
TYPE ProjectRecord
   DECLARE Title : STRING
   DECLARE Score : INTEGER
   DECLARE Submitted : BOOLEAN
ENDTYPE
// Supplied: Projects : ARRAY[1:20] OF ProjectRecord
// Supplied: N : INTEGER
DECLARE Index, BestIndex, BestScore : INTEGER
BestIndex ← 0
BestScore ← -1
FOR Index ← 1 TO N
   IF Projects[Index].Submitted THEN
      IF Projects[Index].Score > BestScore THEN
         BestScore ← Projects[Index].Score
         BestIndex ← Index
      ENDIF
   ENDIF
NEXT Index
IF BestIndex = 0 THEN
   OUTPUT "No submissions"
ELSE
   OUTPUT Projects[BestIndex].Title,
          Projects[BestIndex].Score
ENDIF
```

BestScore = -1 is safe because valid scores are non-negative. The best index remains zero when no submitted project is visited. Strictly greater comparison preserves the first eligible project on a tie.

Test input / state	Expected result
(Map,70,FALSE), (Game,60,TRUE), (Art,60,TRUE)	Game, 60
N = 0, or all Submitted = FALSE	No submissions

Suggested marking points	Marks
Record type with all three fields and types	1
Initialise best-index and best-score state	1
Visit only the N used records	1
Ignore unsubmitted projects	1
Correct comparison preserving first tie	1
Update both winning fields	1
Handle no submissions	1
Output winning title and score	1

Common error: Selecting the largest score before checking Submitted can choose an ineligible project.

PRACTISE / ADVANCED TASK

13 / Recursive digit total

A2 extension • recursion • 8 suggested marks

Write a recursive function DigitTotal(N : INTEGER) RETURNS INTEGER for N >= 0. It must return the sum of the decimal digits of N, without strings or loops. Use a base case for N < 10. Trace the calls and returned values for DigitTotal(5072). Explain why recursion terminates.

Planning prompt

Use N MOD 10 for the last digit and N DIV 10 for the remaining digits.

Algorithm and explanation

Test or trace: include the state before and after each operation.

PRACTISE / ADVANCED TASK

14 / Safe stack removal

A2 extension • stack • BYREF • 8 suggested marks

Stack is a global ARRAY[1:6] OF INTEGER and Top is a global INTEGER. Top = 0 means empty; otherwise Stack[Top] is the top item. Write Pop(BYREF Item : INTEGER, BYREF Success : BOOLEAN) as a procedure. If empty, set Success to FALSE and leave Item and Top unchanged. Otherwise copy the top item to Item, decrease Top, and set Success to TRUE. Show two calls when Top = 1 and Stack[1] = 42.

Planning prompt

Use the precondition 0 <= Top <= 6. Only access Stack[Top] after ruling out an empty stack.

Algorithm and explanation

Test or trace: include the state before and after each operation.

PRACTISE / ADVANCED TASK

15 / Circular queue insertion

A2 extension • queue • wraparound • 9 suggested marks

Queue is a global ARRAY[1:5] OF STRING. Global INTEGERs Front, Rear, and Count describe it: Front is the next item to remove, Rear is the next free slot to fill, and Count is the number stored. Initially Front = 1, Rear = 1, Count = 0. Write Enqueue(Value : STRING, BYREF Success : BOOLEAN). If full, report failure and change no queue state. Otherwise store Value, advance Rear with wraparound, increase Count, and report success. Enqueue must not change Front.

Planning prompt

For a wraparound trace, begin at Front = 4, Rear = 5, Count = 1 with Queue[4] = "old". Insert "new" and state the resulting pointers, count, and slot contents.

Algorithm and explanation

Test or trace: include the state before and after each operation.

PRACTISE / ADVANCED TASK

16 / Build a reference-only file

A2 extension • text files • EOF • 9 suggested marks

The existing text file resources.txt contains one resource code per line. Copy every line beginning with the exact uppercase prefix REF- into a new text file reference.txt, in the same order. Ignore shorter and non-matching lines. Output the number copied. Handle an empty source file. You may assume file operations succeed. LENGTH(S) gives the length and MID(S, Start, Count) gives a substring from position 1.

Planning prompt

Check that a line has at least four characters before taking its four-character prefix. Opening reference.txt in WRITE mode replaces any existing contents.

Algorithm and explanation

Test or trace: include the state before and after each operation.

REVIEW / ADVANCED WORKED SOLUTION

13 / Recursive digit total

A2 extension • recursion • 8 suggested marks

```
FUNCTION DigitTotal(N : INTEGER) RETURNS INTEGER
   IF N < 10 THEN
      RETURN N
   ELSE
      RETURN (N MOD 10) + DigitTotal(N DIV 10)
   ENDIF
ENDFUNCTION
```

Calls descend through 5072, 507, 50, and 5. The base call returns 5. Unwinding gives 0 + 5 = 5, then 7 + 5 = 12, then 2 + 12 = 14. For each recursive call with N >= 10, integer division by 10 produces a smaller non-negative integer; it eventually reaches a single digit.

Test input / state	Expected result
DigitTotal(0)	0
DigitTotal(5072)	14
DigitTotal(999)	27

Suggested marking points	Marks
Typed recursive function signature	1
Correct single-digit base condition	1
Base case returns N	1
Extract final digit using MOD	1
Recursive call on N DIV 10	1
Return the sum	1
Correct call and return trace for 5072	1
Termination explained using a decreasing input	1

Common error: Calling DigitTotal(N) again leaves the argument unchanged and never reaches the base case.

REVIEW / ADVANCED WORKED SOLUTION

14 / Safe stack removal

A2 extension • stack • BYREF • 8 suggested marks

```
PROCEDURE Pop(BYREF Item : INTEGER,
              BYREF Success : BOOLEAN)
   IF Top = 0 THEN
      Success ← FALSE
   ELSE
      Item ← Stack[Top]
      Top ← Top - 1
      Success ← TRUE
   ENDIF
ENDPROCEDURE

DECLARE Value : INTEGER
DECLARE OK : BOOLEAN
CALL Pop(Value, OK)
OUTPUT Value, OK
CALL Pop(Value, OK)
OUTPUT Value, OK
```

Starting from the specified state, the first call outputs 42, TRUE and leaves Top = 0. The second outputs 42, FALSE and leaves Top = 0. Value retains its earlier value after the failed call. BYREF allows Item and Success to update the variables passed by the caller. Global Stack and Top are supplied state, so no local redeclarations are needed.

Test input / state	Expected result
Top 1; Stack[1] 42; two pops	(42, TRUE), then (42, FALSE)
Top 0; Item initially 9	Item 9; Success FALSE; Top 0

Suggested marking points	Marks
Procedure with both BYREF parameters	1
Check empty before array access	1
Report failure on empty	1
Preserve Item and Top on empty	1
Read the current top item	1
Decrease Top and report success	1
Two correctly formed procedure calls	1
Correct state and outputs for both calls	1

Common error: Decreasing Top before reading the item removes the wrong element and can access index 0.

REVIEW / ADVANCED WORKED SOLUTION

15 / Circular queue insertion

A2 extension • queue • wraparound • 9 suggested marks

```
PROCEDURE Enqueue(Value : STRING,
                  BYREF Success : BOOLEAN)
   IF Count = 5 THEN
      Success ← FALSE
   ELSE
      Queue[Rear] ← Value
      Rear ← Rear + 1
      IF Rear > 5 THEN
         Rear ← 1
      ENDIF
      Count ← Count + 1
      Success ← TRUE
   ENDIF
ENDPROCEDURE
```

Rear always identifies the next insertion slot. With Front = 4, Rear = 5 and Count = 1, insertion writes new to Queue[5], wraps Rear to 1, and makes Count = 2. Front stays 4 and Queue[4] stays old. With Count = 5 the failure branch performs no writes to Queue, Front, Rear, or Count.

Test input / state	Expected result
Rear 5; Count 1; insert new	Queue[5] new; Rear 1; Count 2; success
Count 5; any insertion	Failure; all queue state unchanged

Suggested marking points	Marks
Typed procedure and BYREF status	1
Correct full condition	1
Failure status without changing queue state	1
Store at the current Rear	1
Advance Rear	1
Wrap from 6 to 1	1
Increase Count and report success	1
Preserve Front	1
Correct wraparound trace	1

Common error: Front = Rear can describe either empty or full here; Count distinguishes those states.

REVIEW / ADVANCED WORKED SOLUTION

16 / Build a reference-only file

A2 extension • text files • EOF • 9 suggested marks

```
DECLARE Line : STRING
DECLARE Copied : INTEGER
Copied ← 0
OPENFILE "resources.txt" FOR READ
OPENFILE "reference.txt" FOR WRITE
WHILE NOT EOF("resources.txt")
   READFILE "resources.txt", Line
   IF LENGTH(Line) >= 4 THEN
      IF MID(Line, 1, 4) = "REF-" THEN
         WRITEFILE "reference.txt", Line
         Copied ← Copied + 1
      ENDIF
   ENDIF
ENDWHILE
CLOSEFILE "resources.txt"
CLOSEFILE "reference.txt"
OUTPUT Copied
```

EOF is checked before each read, including the first one, so an empty input produces an empty output file and a count of zero. The nested length check keeps the substring call valid without depending on short-circuit evaluation. Writing immediately preserves the source order.

Test input / state	Expected result
Lines: REF-7, AB, ref-8, REF-2	Output lines REF-7, REF-2; count 2
Empty source file	Empty output file; count 0
Line REF-	Line copied; count 1

Suggested marking points	Marks
Open input for READ and output for WRITE	1
Initialise count and use STRING line storage	1
EOF tested before reading	1
Read each source line once	1
Guard substring with a length check	1
Test the exact uppercase prefix	1
Write matches and increase count	1
Close both files	1
Output count, including zero for empty input	1

Common error: APPEND mode retains old output, so rerunning the program would duplicate earlier results.

LEARN / PRACTISE / REVIEW

Teacher guidance

Suggested teaching sequence

Use tasks 1–4 to establish sequence, selection and controlled input. Use 5–7 for array and string traversal. Follow with 8–10 for search and sort, then 11–12 for modular design and record access. Teach the state conventions explicitly before assigning advanced stack and queue tasks.

Assess the algorithm's behaviour

The marking points are independent classroom suggestions. Award each point once when the relevant behaviour is correct. Accept alternative correct algorithms unless a task specifies the method, such as bubble sort, binary search or recursion. Do not award a correct-output point for an algorithm that only happens to work on the displayed example.

Use targeted feedback

Observed issue	Useful follow-up
Uninitialised accumulator	Ask for the value before the first input
Off-by-one error	Trace N = 0, N = 1 and the final used index
Incorrect threshold	Test the boundary and values either side
Sentinel included in data	Mark exactly when the stopping input is tested
Incorrect empty/full operation	Record state before and after a failed call
Non-terminating search/recursion	Identify the quantity that must get smaller

Differentiate with purpose

Support: supply declarations, a trace-table heading, or the first loop condition. Ask students to explain each state change aloud. Extend: require an invariant, compare linear and binary search conditions, or derive the worst-case number of passes for a sorting method.

Feedback record

One strength: _____

One correction: _____

A test that exposed it: _____

LEARN / PRACTISE / REVIEW

References and next steps

Official notation references

The official guides were consulted for notation: assignment arrows, array bounds, NEXT, function return syntax, BYREF parameters and sequential file commands. The teaching explanations, scenarios, solutions and marking points in this workbook are original.

For examinations in 2026:
Cambridge International 9618 Pseudocode Guide for Teachers (2026)

For examinations in 2027–2029:
Cambridge International 9618 Pseudocode Guide (2027–2029)

Qualification and syllabus information:
Cambridge International AS & A Level Computer Science (9618)

Use the syllabus and notation guide for your own examination year. This resource focuses on selected algorithm skills; it does not claim to cover every assessed topic.

Independent resource statement

This workbook was created in response to a request for a new pseudocode resource. It is not a copy, transcription or reformatting of the linked Scribd publication. It contains no official Cambridge past-paper questions or official mark schemes. Cambridge International and other names belong to their respective owners.

Continue your practice

Choose one solution and translate it into your programming language. Run every listed test and add one new edge case. Then change one requirement, update the algorithm, and explain which statements must change and why.

Personal review

A structure I can now explain: _____

A boundary I must remember: _____

My next practice target: _____

Editable source: workbook.json and build.py. Rebuild instructions are in README.md. Prepared November 2026.